241901075
Nishanth Elango Rajan

EX: 4 -DEVELOP A CUSTOMIZED PING COMMAND TO TEST THE SERVER CONNECTIVITY

AIM:
To develop a customized Python-based ping command that tests server connectivity by measuring Round Trip Time (RTT) using socket programming.

ALGORITHM:
Initialize Parameters:

- Define the target host (e.g., google.com) and port (e.g., 80 for HTTP).
- Set the number of ping attempts.

Ping Loop:

- For each attempt:
  - Create a socket connection.
  - Record the start time.
  - Connect to the server.
  - Record the end time.
  - Calculate RTT as the difference between end and start time.
  - Store RTT values for analysis.
  - Print the RTT for each attempt.

Analyze Results:

- After all attempts, compute:
  - Minimum RTT
  - Maximum RTT
  - Average RTT

Display Summary:

- Print the computed RTT statistics.

OUTPUT:

```
C:\Users\tcs\Desktop>python3 ping.py
Reply from google.com: time=94.02 ms
Reply from google.com: time=0.00 ms
Reply from google.com: time=0.00 ms
Reply from google.com: time=15.51 ms
Reply from google.com: time=0.00 ms
Reply from google.com: time=0.00 ms
Reply from google.com: time=0.00 ms
Reply from google.com: time=0.00 ms

Min RTT = 0.0 ms
Max RTT = 0.0 ms
Avg RTT = 0.0 ms

C:\Users\tcs\Desktop>
```

CODE:

```
import socket
import time

host = "google.com"   # you can change this
port = 80             # HTTP port
count = 4             # number of pings

for i in range(count):
    try:
        s = socket.socket()
        start = time.time()
        s.connect((host, port))
        end = time.time()
        s.close()
        print(f"Reply from {host}: time={(end-start)*1000:.2f} ms")
    except Exception:
        print("Request timed out")


import socket, time

host = "google.com"
port = 80
count = 4
times = []

for i in range(count):
    try:
```

```
        s = socket.socket()
        start = time.time()
        s.connect((host, port))
        end = time.time()
        s.close()
        rtt = (end - start) * 1000
        times.append(rtt)
        print(f"Reply from {host}: time={rtt:.2f} ms")
    except:
        print("Request timed out")

if times:
    print("\nMin RTT =", min(times), "ms")
    print("Max RTT =", max(times), "ms")
    print("Avg RTT =", sum(times)/len(times), "ms")
```

RESULT:

The program successfully connects to the specified server multiple times and prints the RTT for each attempt. It also calculates and displays the minimum, maximum, and average RTT values, providing a basic diagnostic of network latency.